

ROADRESQ

Backend Schema Document

Version 1.0
FastAPI + PostgreSQL/PostGIS + Redis + WebSockets

AUTH → REQUEST → MATCH → TRACK → JOB → BILL → PAY → REVIEW

Backend architecture, API contracts, domain services, background jobs, realtime events, security, and deployment blueprint.

Document	RoadResQ Backend Schema
Version	1.0
Backend	FastAPI / Python
Database	PostgreSQL + PostGIS
Realtime / Cache	Redis + WebSockets
Clients	Customer App, Provider App, Admin Panel

1. Backend Objectives

The RoadResQ backend owns authentication, authorization, booking lifecycle, provider matching, live tracking, job execution, estimates, invoices, payments, notifications, support, and auditability.

Requirement	Responsibility
API	Versioned REST APIs for clients.
Business logic	Validate workflows and state transitions.
Dispatch	Find, rank, notify, and assign suitable providers.
Realtime	Publish active booking/location events.
Persistence	PostgreSQL/PostGIS as source of truth.
Async work	Redis-backed workers for notifications, retries, and scheduled tasks.
Security	RBAC, object authorization, rate limiting, audit logging.

2. Recommended Stack

Layer	Technology	Role
API	FastAPI + Python	REST, validation, dependency injection.
ORM	SQLAlchemy 2.x	Models and queries.
Validation	Pydantic	Request/response contracts.
Database	PostgreSQL	Transactional data.
Geospatial	PostGIS	Nearby-provider queries.
Cache/Queue	Redis	Caching, locks, queues, rate limits.
Realtime	WebSockets	Live updates.
Workers	Celery/RQ/Arq	Background tasks.
Storage	S3-compatible	Documents/media.
Migrations	Alembic	Versioned DB changes.

3. Domain Modules

Module	Responsibility
Auth	OTP/login, tokens, sessions, roles.
Users	Profiles and addresses.
Vehicles	Vehicle CRUD and ownership.
Providers	Profile, verification, services, availability.
Dispatch	Matching, offers, timeout/retry, assignment.
Bookings	Creation and lifecycle.
Jobs	Inspection, estimates, approval, work.
Billing	Invoices, payments, payouts.
Notifications	Push/SMS/email/in-app.
Support	Tickets, disputes, audit.

4. Project Structure

```
app/ main.py core/ # config, security, logging, exceptions db/ # engine, session, migrations models/ # SQLAlchemy models
schemas/ # Pydantic schemas api/v1/ # versioned routes services/ # business/domain services repositories/ # data-access layer
workers/ # background jobs websocket/ # realtime handlers integrations/ # maps, payments, SMS, push, storage middleware/ #
auth, request-id, rate limits tests/ # unit/integration/API tests
```

5. API Standards

Rule	Standard
Base path	/api/v1
Errors	code, message, details, request_id.
Pagination	Page/page_size or cursor.
Idempotency	Required for payments and critical creation.
Tracing	X-Request-ID.
Dates	ISO 8601 / UTC.
Money	Decimal + currency.

6. Authentication & Authorization

OTP/Login → Access + Refresh Token → Middleware → User Context → RBAC → Object Authorization → Route

Role	Permissions
CUSTOMER	Own profile, vehicles, bookings, payments, reviews, support.
PROVIDER	Provider profile, requests, jobs, estimates, earnings.
ADMIN	Platform management, verification, support, analytics, audit.

7. Authentication API

Request OTP

Method	Endpoint	Purpose
POST	/api/v1/auth/otp/request	Start phone verification.

Auth: Public

Request: phone

Response: request_id, expires_in

Errors: 400; 429

Verify OTP

Method	Endpoint	Purpose
POST	/api/v1/auth/otp/verify	Verify OTP and issue session.

Auth: Public

Request: phone, otp

Response: access_token, refresh_token, user

Errors: 400

Refresh

Method	Endpoint	Purpose
POST	/api/v1/auth/refresh	Refresh access token.

Auth: Refresh token

Request: refresh_token

Response: access_token

Errors: 401

Logout

Method	Endpoint	Purpose
POST	/api/v1/auth/logout	End session.

Auth: Authenticated

Request: session context

Response: success

Errors: 401

Current User

Method	Endpoint	Purpose
GET	/api/v1/auth/me	Return current user.

Auth: Authenticated

Request: None

Response: user

Errors: 401

8. Customer & Vehicle API

Profile

Method	Endpoint	Purpose
GET	/api/v1/users/me	Get profile.

Auth: Customer

Request: None

Response: user

Errors: 401

Update Profile

Method	Endpoint	Purpose
PATCH	/api/v1/users/me	Update profile.

Auth: Customer

Request: profile fields

Response: user

Errors: 400/409

List Vehicles

Method	Endpoint	Purpose
GET	/api/v1/vehicles	List vehicles.

Auth: Customer

Request: filters/page

Response: vehicles

Errors: 401

Add Vehicle

Method	Endpoint	Purpose
POST	/api/v1/vehicles	Register vehicle.

Auth: Customer

Request: vehicle fields

Response: vehicle

Errors: 400/409

Vehicle Detail

Method	Endpoint	Purpose
GET	/api/v1/vehicles/{vehicle_id}	View vehicle.

Auth: Customer

Request: None

Response: vehicle

Errors: 403/404

Service History

Method	Endpoint	Purpose
GET	/api/v1/vehicles/{vehicle_id}/history	View service history.

Auth: Customer

Request: pagination

Response: history

Errors: 403/404

9. Provider API

Registration

Method	Endpoint	Purpose
POST	/api/v1/providers	Create provider profile.

Auth: Provider

Request: business details

Response: provider + verification

Errors: 400

Public Profile

Method	Endpoint	Purpose
GET	/api/v1/providers/{provider_id}	View provider.

Auth: Authenticated

Request: None

Response: profile

Errors: 404

Update Profile

Method	Endpoint	Purpose
PATCH	/api/v1/providers/me	Update provider.

Auth: Provider

Request: business fields

Response: provider

Errors: 400/403

Services

Method	Endpoint	Purpose
PUT	/api/v1/providers/me/services	Configure services/prices.

Auth: Provider

Request: service IDs + prices

Response: services

Errors: 400

Availability

Method	Endpoint	Purpose
PUT	/api/v1/providers/me/availability	Set work windows.

Auth: Provider

Request: weekly schedule

Response: availability

Errors: 400

Online State

Method	Endpoint	Purpose
POST	/api/v1/providers/me/online	Set online/offline.

Auth: Provider

Request: is_online

Response: state

Errors: 409

Documents

Method	Endpoint	Purpose
POST	/api/v1/providers/me/documents	Submit verification document.

Auth: Provider

Request: type + storage ref

Response: document

Errors: 400

10. Booking & Emergency API

Create Emergency Request

Method	Endpoint	Purpose
POST	/api/v1/bookings	Create roadside request.

Auth: Customer

Request: vehicle, service, location, problem, media

Response: booking

Errors: 400/409

Booking Detail

Method	Endpoint	Purpose
GET	/api/v1/bookings/{booking_id}	Get booking aggregate.

Auth: Participant

Request: None

Response: booking + provider

Errors: 403/404

My Bookings

Method	Endpoint	Purpose
GET	/api/v1/bookings	List customer bookings.

Auth: Customer

Request: status/date/page

Response: bookings

Errors: 401

Cancel

Method	Endpoint	Purpose
POST	/api/v1/bookings/{booking_id}/cancel	Cancel eligible booking.

Auth: Participant

Request: reason

Response: booking

Errors: 409

Provider Requests

Method	Endpoint	Purpose
GET	/api/v1/provider/requests	List dispatch offers.

Auth: Provider

Request: status/page

Response: offers

Errors: 403

Accept Request

Method	Endpoint	Purpose
POST	/api/v1/provider/requests/{booking_id}/accept	Atomically accept offer.

Auth: Provider

Request: optional ETA

Response: booking

Errors: 409

Reject Request

Method	Endpoint	Purpose
POST	/api/v1/provider/requests/{booking_id}/reject	Reject offer.

Auth: Provider

Request: reason

Response: offer

Errors: 409

Provider Location

Method	Endpoint	Purpose
POST	/api/v1/provider/location	Send GPS point.

Auth: Provider

Request: lat/lng/accuracy/booking

Response: ack

Errors: 400

11. Booking State Machine

Current	Allowed next states
REQUESTED	SEARCHING, CANCELLED
SEARCHING	PROVIDER_ASSIGNED, EXPIRED, CANCELLED
PROVIDER_ASSIGNED	ACCEPTED, EXPIRED, CANCELLED
ACCEPTED	ON_THE_WAY, CANCELLED
ON_THE_WAY	ARRIVED, CANCELLED
ARRIVED	INSPECTION, CANCELLED
INSPECTION	ESTIMATE_PENDING, IN_PROGRESS
ESTIMATE_PENDING	APPROVED, CANCELLED, DISPUTED
APPROVED	IN_PROGRESS
IN_PROGRESS	COMPLETED, DISPUTED
COMPLETED	INVOICE_GENERATED
INVOICE_GENERATED	PAYMENT_COMPLETED
PAYMENT_COMPLETED	CLOSED

Every transition validates the current state, updates the booking, appends status history, and publishes a domain event.

12. Job & Estimate API

Create Job

Method	Endpoint	Purpose
POST	/api/v1/bookings/{booking_id}/job	Create work order.

Auth: Provider/System

Request: job metadata

Response: job card

Errors: 409

Start Inspection

Method	Endpoint	Purpose
POST	/api/v1/jobs/{job_id}/inspections	Create inspection.

Auth: Provider

Request: odometer, notes

Response: inspection

Errors: 403/409

Add Finding

Method	Endpoint	Purpose
POST	/api/v1/inspections/{inspection_id}/items	Add finding.

Auth: Provider

Request: component, condition, finding, media

Response: item

Errors: 400

Create Estimate

Method	Endpoint	Purpose
POST	/api/v1/jobs/{job_id}/estimates	Create estimate.

Auth: Provider

Request: line items, taxes, discount

Response: estimate

Errors: 409

Approve Estimate

Method	Endpoint	Purpose
POST	/api/v1/estimates/{estimate_id}/approve	Approve work.

Auth: Customer

Request: confirmation

Response: approved estimate

Errors: 403/409

Update Job

Method	Endpoint	Purpose
POST	/api/v1/jobs/{job_id}/status	Update progress.

Auth: Provider

Request: status, note

Response: job state

Errors: 409

Complete Job

Method	Endpoint	Purpose
POST	/api/v1/jobs/{job_id}/complete	Complete work and trigger invoice.

Auth: Provider

Request: notes/media

Response: completed job

Errors: 409

13. Provider Matching & Dispatch

Stage	Backend logic
1. Candidate query	Find nearby verified/active providers with PostGIS.
2. Capability	Filter requested service.
3. Availability	Exclude offline/unavailable providers.
4. Compatibility	Apply vehicle/service capability.
5. Ranking	Distance/ETA, rating, reliability, workload, acceptance.
6. Offer	Notify candidate with timeout.
7. Lock	Accept in transaction to prevent double assignment.
8. Retry	Select next candidate set after rejection/timeout.

Dispatch belongs in a domain service/background worker rather than the HTTP route.

13.1 Internal Dispatch API

Candidates

Method	Endpoint	Purpose
POST	/api/v1/dispatch/candidates	Find ranked candidates.

Auth: Internal service

Request: booking context

Response: candidate list

Errors: 409

Assign

Method	Endpoint	Purpose
POST	/api/v1/dispatch/{booking_id}/assign	Atomically assign provider.

Auth: Internal service

Request: provider_id

Response: booking

Errors: 409

Retry

Method	Endpoint	Purpose
POST	/api/v1/dispatch/{booking_id}/retry	Retry matching.

Auth: Internal service

Request: None

Response: dispatch state

Errors: 409

14. Billing & Payment API

Get Estimate

Method	Endpoint	Purpose
GET	/api/v1/estimates/{estimate_id}	View estimate.

Auth: Participant

Request: None

Response: estimate

Errors: 403/404

Get Invoice

Method	Endpoint	Purpose
GET	/api/v1/invoices/{invoice_id}	View invoice.

Auth: Participant

Request: None

Response: invoice

Errors: 403/404

Create Payment

Method	Endpoint	Purpose
POST	/api/v1/payments	Start payment attempt.

Auth: Customer

Request: invoice, amount, method, idempotency key

Response: payment

Errors: 400/409

Gateway Webhook

Method	Endpoint	Purpose
POST	/api/v1/payments/webhook	Receive payment event.

Auth: Gateway

Request: signed event

Response: ack

Errors: 400 signature

Payment Status

Method	Endpoint	Purpose
GET	/api/v1/payments/{payment_id}	Check payment.

Auth: Customer/Admin

Request: None

Response: payment

Errors: 403/404

Provider Earnings

Method	Endpoint	Purpose
GET	/api/v1/providers/me/earnings	Get earnings.

Auth: Provider

Request: date range/page

Response: earnings

Errors: 403

15. WebSocket / Realtime

REST remains the source of truth. WebSockets provide live events for active bookings.

Channel	Audience	Events
booking:{id}	Customer + provider	status_changed, assigned, arrived, estimate_created, approved, job_updated, invoice_created, payment_updated.
provider:{id}	Provider devices	new_request, request_expired, booking_cancelled.
user:{id}	Authenticated user	notification_created, support_update.

Connection: authenticate → authorize → connect → subscribe → receive events → reconnect → fetch REST snapshot → resume.

16. Background Worker Jobs

Job	Trigger	Action
dispatch_request	Booking created	Find/notify providers.
dispatch_timeout	Offer timeout	Expire and retry.
provider_eta	Location update	Recalculate ETA.
send_notification	Domain event	Push/SMS/email/in-app.

Job	Trigger	Action
generate_invoice	Job completed	Create invoice.
payment_reconciliation	Gateway event/schedule	Reconcile payment.
provider_payout	Payment completed	Create payout.
cleanup_location	Scheduled	Apply GPS retention.
analytics_rollup	Scheduled	Aggregate KPIs.

17. Admin & Support API

Dashboard

Method	Endpoint	Purpose
GET	/api/v1/admin/dashboard	Platform KPIs.

Auth: Admin

Request: date filters

Response: metrics

Errors: 403

Users

Method	Endpoint	Purpose
GET	/api/v1/admin/users	Search/manage users.

Auth: Admin

Request: filters/page

Response: users

Errors: 403

Providers

Method	Endpoint	Purpose
GET	/api/v1/admin/providers	Manage providers.

Auth: Admin

Request: filters/page

Response: providers

Errors: 403

Verification

Method	Endpoint	Purpose
POST	/api/v1/admin/providers/{id}/verification	Approve/reject provider.

Auth: Admin

Request: decision, note

Response: state

Errors: 403/409

Bookings

Method	Endpoint	Purpose
GET	/api/v1/admin/bookings	Monitor bookings.

Auth: Admin

Request: filters/page

Response: bookings

Errors: 403

Support

Method	Endpoint	Purpose
GET	/api/v1/admin/support/tickets	Manage support.

Auth: Admin

Request: status/priority/page

Response: tickets

Errors: 403

Audit

Method	Endpoint	Purpose
GET	/api/v1/admin/audit-logs	Search audit events.

Auth: Admin

Request: entity/date/page

Response: audit logs

Errors: 403

18. Pydantic Schema Layers

Layer	Examples	Purpose
Create	BookingCreate, VehicleCreate, ProviderCreate	Validate client input.
Update	VehicleUpdate, ProviderUpdate, BookingCancel	Validate changes.
Read	BookingRead, ProviderRead, InvoiceRead	Stable responses.
Nested	BookingWithProvider, JobWithEstimate	Aggregate resources.
Internal	DispatchCandidate, PaymentEvent	Service/worker contracts.
Event	BookingStatusChanged, ProviderLocationUpdated	Realtime/outbox payloads.

19. Security Architecture

Control	Implementation
Authentication	Short-lived access tokens, refresh rotation/session revocation.
Authorization	RBAC + object-level ownership/assignment checks.
Rate limiting	Redis-backed limits on OTP, auth, booking, payment, messaging.
Validation	Pydantic + DB constraints; validate coordinates/uploads.
Secrets	Environment/secret manager; never commit credentials.
Payments	Verify webhook signatures; use idempotency keys.
Uploads	Private storage, validation, scanning.
Logging	Never log passwords, OTPs, tokens, or sensitive payment data.
Audit	Record sensitive admin/provider/payment changes.

20. Error Contract

HTTP	Meaning
400	Malformed/invalid request.
401	Missing/invalid authentication.

HTTP	Meaning
403	Not authorized.
404	Resource not found/hidden.
409	Business conflict/duplicate.
422	Schema validation.
429	Rate limited.
500	Unexpected server error + request ID.

21. Database Integration Rules

Rule	Implementation
Source of truth	PostgreSQL holds durable state.
Transactions	Assignment, state changes, invoice/payment updates are transactional.
Spatial	PostGIS + GiST for nearby provider queries.
Cache	Redis only for rebuildable/expiring data.
Concurrency	Locks and idempotency prevent duplicate acceptance/payment.
Migrations	Alembic versioned migrations.
Repositories	Keep queries out of route handlers.

22. Deployment Blueprint

Component	Role
API containers	FastAPI replicas behind reverse proxy/load balancer.
Workers	Background task execution.
PostgreSQL	Primary DB with backups/PITR.
Redis	Cache, queues, locks, rate limits.
Object storage	Private documents/media.
Monitoring	Logs, metrics, tracing, error tracking.
CI/CD	Tests, migrations, build, deployment.

23. Environment Variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection.
REDIS_URL	Redis connection.
JWT_SECRET / key set	Token signing.
STORAGE_BUCKET	Private object storage.
MAPS_API_KEY	Maps/geocoding/ETA.
PAYMENT_SECRET	Payment credentials/webhook verification.
SMS/PUSH credentials	Notification providers.
CORS_ORIGINS	Allowed client origins.

24. Testing Strategy

Layer	Coverage
Unit	Matching, state transitions, pricing, authorization.
Integration	PostgreSQL/PostGIS, Redis, workers.
API	Auth, CRUD, lifecycle, roles, validation.
Concurrency	Two providers accepting one job; retries; duplicate webhooks.
Realtime	WebSocket authorization, reconnect, event delivery.
E2E	Request → match → arrival → inspection → approval → completion → payment.

25. Critical End-to-End Flow

Customer auth → vehicle → **POST /bookings** → **REQUESTED** → **dispatch** → **provider offer** → **atomic acceptance** → **live tracking** → **arrival OTP** → **inspection** → **estimate** → **approval** → **work** → **completion** → **invoice** → **payment** → **payout** → **review** → **CLOSED**.

26. Backend Completion Checklist

Area	Definition of done
Auth	OTP/login, refresh, logout, RBAC, session controls.
Dispatch	Spatial matching, timeout/retry, atomic assignment.
Realtime	Secure active-booking status/location events.
Job	Inspection, estimate, approval, completion.
Billing	Invoice, payment, webhook, reconciliation, payout.
Admin	Verification, booking monitoring, support, audit.
Reliability	Retries, idempotency, logs, metrics, backups, migrations.
Security	Object authorization, rate limits, secure uploads, secret management.

27. Final Backend Schema Summary

RoadResQ should begin as a modular FastAPI backend with PostgreSQL/PostGIS as the durable source of truth and Redis for queues, locks, caching, rate limits, and short-lived state. Domain services own business rules while route handlers remain thin.

The architecture supports the MVP and provides a clean path toward garage management, fleet operations, AI diagnosis, parts, insurance, EV services, subscriptions, and a larger provider network.